

Here are the **model answers** for the top 5 highest-yield questions from your analysis, structured exactly as a top-scoring Mumbai University student would write them. Referred from this :

[Important Concepts and Questions for BDT](#)

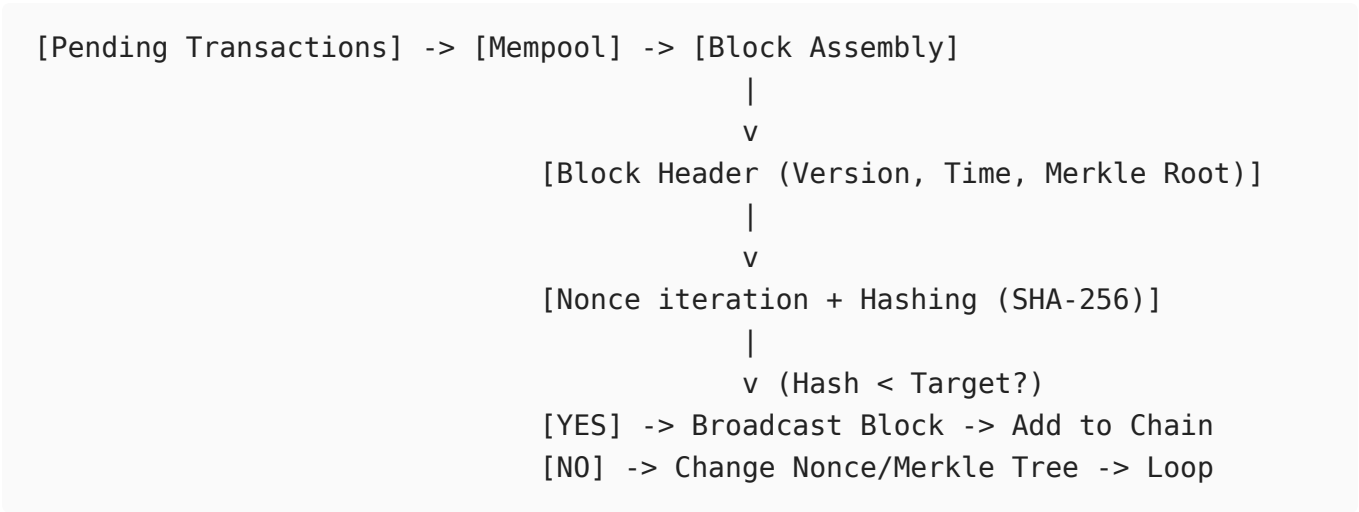
I have followed the **"Sharp Mind" format** you requested: Diagrams where possible, tables for differentiation, step-by-step mechanics, and bold keywords.

Question 1: Explain Bitcoin Mining process. What is difficulty level and how is it calculated? (10 Marks)

1. Definition & Purpose (2 lines)

Bitcoin Mining is the process of validating new transactions and adding them to the blockchain ledger. It serves two purposes: creating new bitcoins (coinbase reward) and securing the network through Proof-of-Work (PoW).

2. Architectural Diagram (Crucial for 10 Marks)



3. Step-by-Step Mechanism (Numbered List)

- Transaction Verification:** Node verifies signatures and checks UTXOs to prevent double-spending.
- Mempool Selection:** Miner selects high-fee transactions from the memory pool.
- Block Construction:** Creates a candidate block with a previous block hash and a **Merkle Root**.
- Nonce Discovery (PoW):** Miner changes the **Nonce** (32-bit number) and hashes the block header repeatedly using SHA-256.

5. **Target Condition:** The resulting hash must be *less than* the current **Difficulty Target** (leading zeros).
6. **Broadcast & Validation:** The winning block is broadcast; other nodes verify it before appending.

4. Difficulty Calculation (Mathematical Logic)

- **Purpose:** Ensures block time remains constant at ~10 minutes, regardless of total network hashrate.
- **Formula:**

$$\text{New Difficulty} = \text{Old Difficulty} \times (\text{Actual Time for 2016 blocks} / \text{Expected Time for 2016 blocks})$$

- *Expected Time:* 2016 blocks × 10 minutes = 20,160 minutes (2 weeks).
- *Adjustment:* Recalculated every 2016 blocks. If miners are faster → Difficulty ↑. If slower → Difficulty ↓.

5. Significance

- **Security:** Makes rewriting history computationally infeasible.
- **Monetary Policy:** Controls Bitcoin inflation (halving every 210,000 blocks).

Question 2: Differentiate PoW vs PoS consensus algorithms. (5 Marks)

Parameter	Proof of Work (PoW)	Proof of Stake (PoS)
Energy Consumption	Extremely high (ASIC mining farms)	Very low (Virtually negligible)
Validator Selection	Based on computational power (Hashrate)	Based on stake amount (coins locked) & randomisation
Hardware Requirement	Specialized ASIC/GPUs	Standard laptop/validator node
51% Attack Cost	Attack cost = 51% of network hashrate (billions \$)	Attack cost = 51% of total coin supply (impractical)
Block Finality	Probabilistic (can be reversed by longer chain)	Deterministic (validators are slashed if they cheat)
Example	Bitcoin, Litecoin (pre-Merge Ethereum)	Cardano, Solana, Ethereum (post-Merge)

Parameter	Proof of Work (PoW)	Proof of Stake (PoS)
Reward Mechanism	Block reward + transaction fees	Transaction fees only (no new coins created)

Key Keyword to write: *"Nothing-at-Stake problem in PoS vs. Energy waste in PoW."*

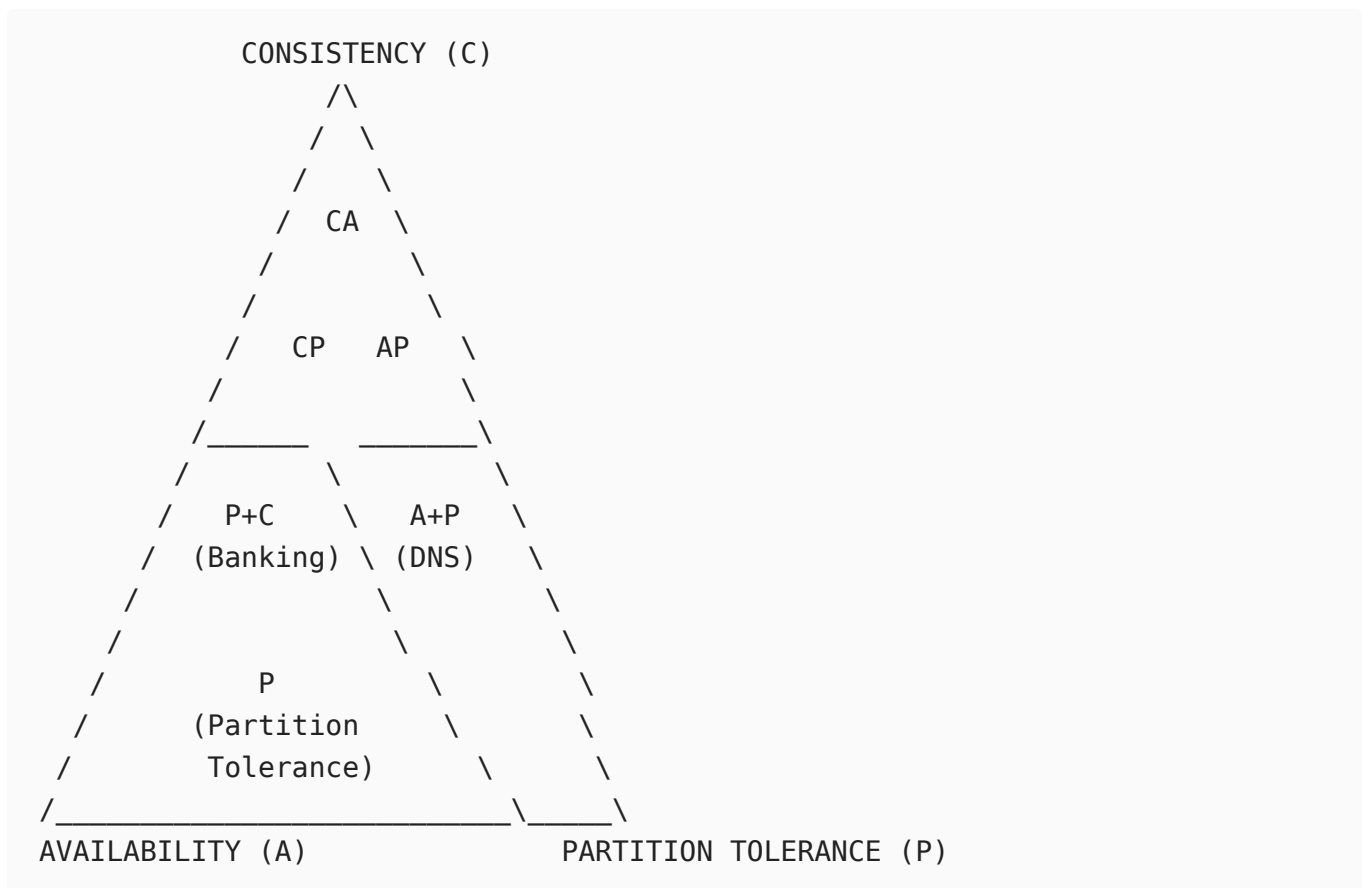
Question 3: Explain CAP Theorem with relationship diagram. (5 Marks)

1. Definition

The CAP Theorem (Brewer's Theorem) states that a distributed data store (including a blockchain network) can provide at most **two** of the following three guarantees simultaneously:

- **C (Consistency):** Every node returns the same data at the same time.
- **A (Availability):** Every request receives a response (without guarantee it is the latest data).
- **P (Partition Tolerance):** The system continues to operate despite network message loss/delay between nodes.

2. Relationship Diagram (Triangle)



3. How this applies to Blockchains:

- **Public Blockchains (Bitcoin/Ethereum):** Choose **C + P** (Consistency + Partition Tolerance). They sacrifice Availability during network splits (forking).
- **Permissioned Blockchains (Hyperledger Fabric):** Often choose **A + P** (Availability + Partition Tolerance) or **C + A** (if network is trusted without partitions).

Memory Hack: *"In a network partition, you must choose: Stop (C+P) or Accept old data (A+P)."*

Question 4: Explain Hyperledger Fabric Components: Peer, Orderer, CA, Chaincode. (10 Marks)

1. High-Level Architecture

Hyperledger Fabric is a **permissioned** blockchain with a modular architecture supporting confidential transactions.

2. Component Deep Dive (Table format for clarity)

Component	Role	Key Characteristics
Peer Node	Maintains ledger & executes chaincode	Two types: Endorsing Peer (simulates tx) & Committing Peer (validates & adds to ledger). Stores World State (LevelDB/CouchDB).
Orderer Node	Consensus & block sequencing	Implements Raft (crash fault-tolerant) or BFT. Decouples transaction ordering from execution (Unique to Fabric). Creates blocks.
Certificate Authority (CA)	Identity management	Issues X.509 certificates for members, peers, and admins. Enables Membership Service Provider (MSP) . Revokes permissions.
Chaincode	Smart contract	Written in Go, Java, or Node.js. Runs inside a Docker container (isolated). Queries/updates ledger via GetState/PutState API.

3. Transaction Flow (Crucial for full marks)

1. **Client** proposes transaction to **Endorsing Peers** (via SDK).
2. **Endorsing Peers** simulate chaincode → generate **RW Set** (Read-Write set) → sign & return.
3. **Client** sends RW Set + endorsements to **Orderer**.
4. **Orderer** creates block (Raft consensus) → broadcasts block to all **Committing Peers**.

5. **Committing Peers** validate endorsements policy & MVCC check (Multi-Version Concurrency Control) → update ledger.

4. Why this matters.

- **Privacy:** Only relevant peers see specific chaincode data (via Channels).
- **Performance:** No wasteful mining (PoW).

Question 5: What is a Block? Explain Block Header, Genesis Block, Linking Blocks, Merkle Tree. (10 Marks)

1. Definition of a Block

A Block is a data structure representing a batch of validated transactions bundled together, cryptographically linked to the previous block, forming the blockchain.

2. Block Structure Diagram

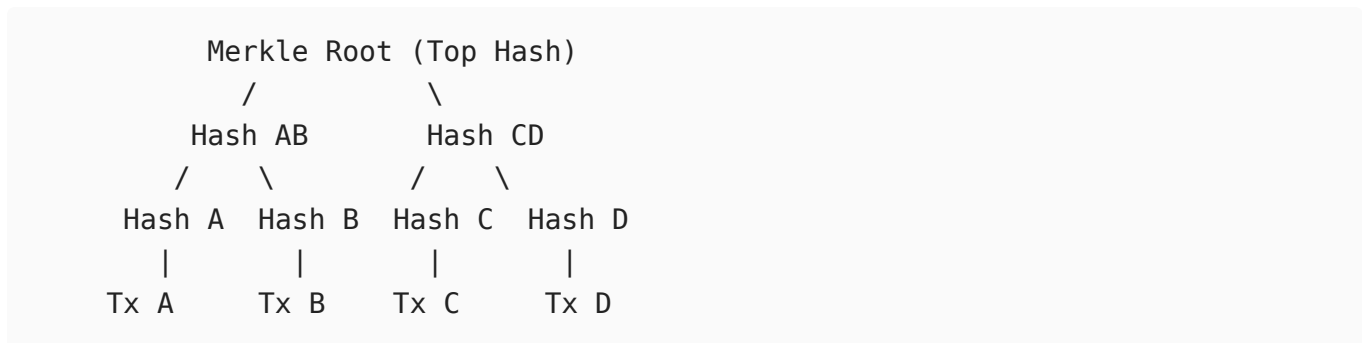
```
[BLOCK  #NODE_ID_VERIFY]
+-----+
|  MAGIC_NO  | BLOCKSIZE  |
+-----+
|           BLOCK HEADER           |
| +-----+ |
| | Version | Timestamp | |
| | Previous Block Hash (Pointer) | |
| | Merkle Root (of transactions) | |
| | Nonce   | Difficulty Target | |
| +-----+ |
+-----+
|           BLOCK BODY           |
| +-----+ |
| | Transaction Counter (Tx_Count)| |
| | List of Transactions (Tx1..Txn)| |
| +-----+ |
+-----+
```

3. Explanation of Components

Component	Explanation
Block Header	Metadata about the block. Contains the Previous Block Hash (linking mechanism), Timestamp, Version, Nonce, Bits (Difficulty), and Merkle Root .

Component	Explanation
Genesis Block	The first block of the blockchain (Block 0 or Block 1). Hardcoded into the software. Has no previous block hash (often set to 0x00...00). Example: Bitcoin Genesis Block (2009).
Linking Blocks	Each block contains the cryptographic hash of the previous block's header. This forms a tamper-evident chain . If Block N changes → Hash(N) changes → Block N+1's "Previous Hash" becomes invalid → Chain breaks.
Merkle Tree	A binary hash tree structure. Leaves are transaction hashes. Internal nodes are hashes of child nodes. The final root is the Merkle Root (stored in header). Allows SPV proof (a light node can verify a transaction exists without downloading the entire block).

4. Merkle Tree Visual



5. Significance

- **Integrity:** Any single transaction change changes the Merkle Root → invalidates the block.
- **Efficiency:** Lightweight wallets only store block headers.

Here are the **model answers** for the remaining 10 high-yield questions from your list, structured in the same "Sharp Mind" format (diagrams, tables, step-by-step mechanics, bold keywords).



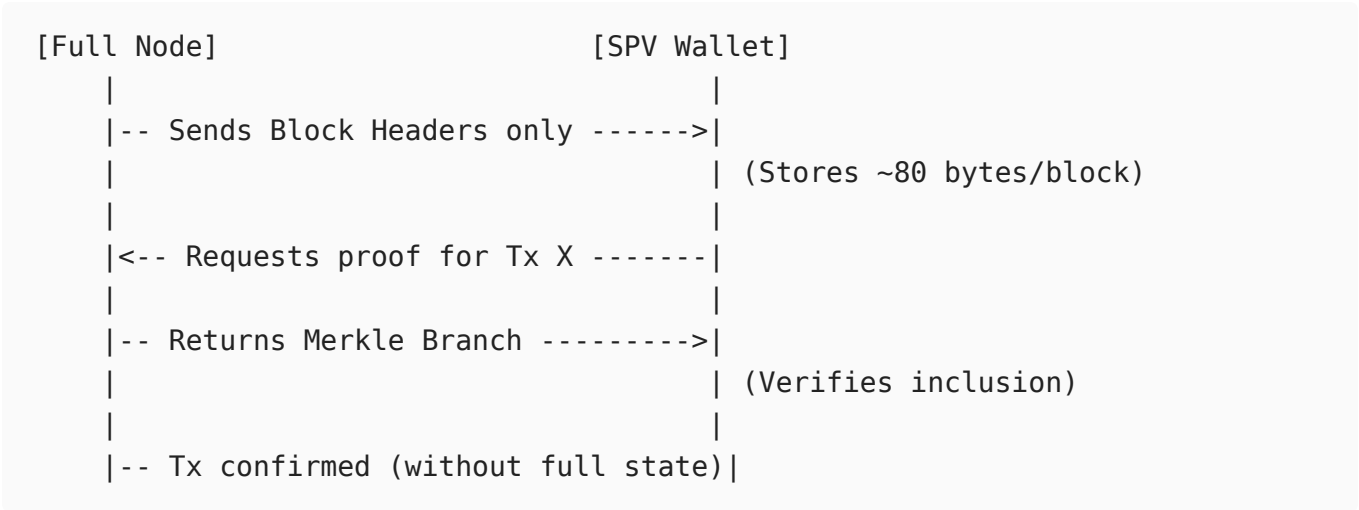
REST OF THE HIGH-YIELD QUESTION BANK

Question 6: Explain SPV nodes and their privacy issues & solution. (5-10 Marks)

1. Definition

SPV (Simplified Payment Verification) nodes, also called "lightweight wallets," download only block headers (80 bytes each) instead of full blocks. They verify transactions using **Merkle proofs** without running a full node.

2. How SPV Works (Step-by-Step)



Mechanism:

- 1. SPV node connects to several full nodes.
- 2. Downloads only block headers (not transactions).
- 3. To verify its own transaction, requests a **Merkle Branch** (log2 N hashes).
- 4. Recomputes Merkle Root from branch + own tx hash → compares with header.

3. Privacy Issues (Critical for 10 Marks)

Privacy Issue	Explanation
IP Address Leakage	SPV node queries full nodes for specific addresses/transactions, revealing which addresses belong to the user.
Bloom Filter Flaws	SPV uses Bloom filters to request relevant tx. Malicious full node can reverse-engineer the filter to identify user's entire wallet.
Network Fingerprinting	Timing analysis of requests can correlate multiple addresses to same user.
Connection Spying	The full node you connect to sees every address you check.

4. Solutions

Solution	How it works
Tor Integration	Route all SPV queries through Tor to hide IP (Samourai Wallet, Wasabi).
Neutrino Protocol	Uses compact filters (Golomb-Rice encoded) instead of Bloom filters → better privacy.
Client-Side Block Filtering	Download block filters, filter locally without revealing addresses to server (BIP 158).
Dandelion Protocol	Propagates requests through random peers before broadcasting, obscuring origin.

Exam Keyword: "Bloom filters are privacy-leaking. Neutrino or client-side filtering is the modern solution."

Question 7: Differentiate Hard Fork vs Soft Fork with examples. (5 Marks)

Parameter	Hard Fork	Soft Fork
Backward Compatibility	✗ NOT backward compatible	✓ Backward compatible
Old Nodes	Reject new blocks (split chain)	Accept new blocks (still valid)
New Feature Example	Increasing block size (4MB → 8MB)	Adding new opcode (e.g., Schnorr signatures)
Valid Blocks	New rules = new types only	New blocks = new rules, old blocks = old rules
Activation	Requires near-100% consensus	Requires >51% hashrate
Result	Two separate blockchains (permanent split)	Single blockchain (temporary fork, then unified)
Real Example	Bitcoin Cash (BTC → BCH, 2017, block size 1MB → 8MB)	SegWit (Bitcoin, 2017, new transaction format)
Another Example	Ethereum → Ethereum Classic (DAO hack 2016)	Taproot (Bitcoin, 2021, new script types)

Visual Representation


```
Chain A: ---[Block5]---[Block6]---[Block7]--- (BTC)
          \
            \---[Block6']---[Block7']--- (BCH) ← New chain
```

```
Chain: ---[OldBlock]---[NewBlock(old nodes see valid)]---[NewBlock]---
```

↑
Old nodes still accept this block

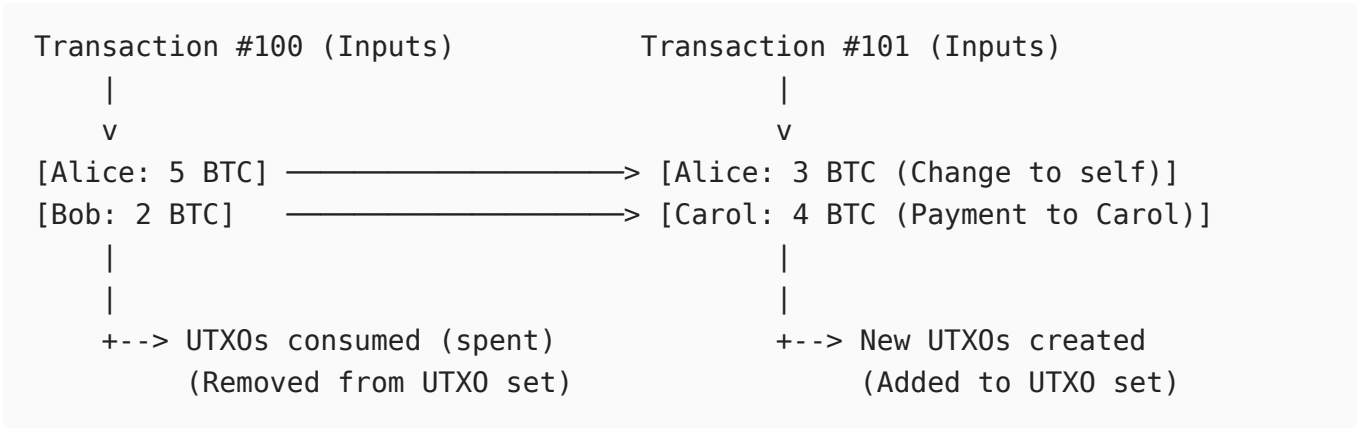
Exam Keyword: "Hard fork = chain split (ETH/ETC). Soft fork = upgrade without split (SegWit)."

Question 8: Explain UTXO model in Bitcoin. How are transaction fees calculated? (10 Marks)

1. Definition

UTXO (Unspent Transaction Output) is Bitcoin's accounting model. Unlike bank accounts (balance), Bitcoin tracks individual "coins" — pieces of value locked to an owner's public key.

2. UTXO Model Diagram



3. How UTXO Works (Step-by-Step)

Step	Action
1	Alice receives 5 BTC from Bob → UTXO created: (5 BTC, Alice's PubKey)
2	Alice wants to pay Carol 3 BTC → Must spend the ENTIRE 5 BTC UTXO
3	Creates transaction with: 2 outputs → (3 BTC to Carol) + (2 BTC change to Alice)

Step	Action
4	The original 5 BTC UTXO is marked spent (removed from UTXO set)
5	Two NEW UTXOs created: (3 BTC, Carol) and (2 BTC, Alice)

4. Transaction Fee Calculation

Formula:

$$\text{Transaction Fee} = \text{Sum}(\text{Input UTXOs}) - \text{Sum}(\text{Output UTXOs})$$

Example:

- Input UTXOs: 1 BTC + 0.5 BTC + 0.3 BTC = **1.8 BTC total**
- Output UTXOs: 1.2 BTC (to merchant) + 0.4 BTC (change to self) = **1.6 BTC total**
- **Fee = 1.8 - 1.6 = 0.2 BTC**

Fee is NOT a separate output. It's the "unclaimed" difference that miners collect.

5. Fee Rate Calculation (Important for high-fee periods)

Term	Formula	Example
Transaction Size	Inputs(148 bytes each) + Outputs(34 bytes each) + overhead	2 inputs + 2 outputs ≈ 372 bytes
Fee Rate (sat/vB)	Fee (sats) / Transaction size (vBytes)	10,000 sats / 250 vBytes = 40 sat/vB
Priority Fee	Higher sat/vB = faster confirmation	10 sat/vB (slow) vs 100 sat/vB (fast)

6. UTXO Advantages & Disadvantages

Advantage	Disadvantage
✓ Parallel spending (no account lock)	✗ UTXO explosion (many small coins)
✓ Privacy (new address per transaction)	✗ Change management complexity
✓ Verification simplicity (no replay attacks)	✗ Larger transaction size

Exam Keyword: "UTXO = cash analogy. Spending a \$50 bill to pay \$3 gives \$47 change. Fee = missing amount."

Question 9: What is Byzantine Generals Problem? How does it relate to consensus? (5 Marks)

1. The Problem Statement

The **Byzantine Generals Problem** describes a scenario where several generals (nodes) surrounding a city must agree on a common attack plan (coordinate), but:

- Some generals may be **traitors** (malicious nodes).
- Messages may be **delayed or corrupted** (unreliable network).
- **Question:** How can loyal generals reach consensus?

2. Visual Analogy

```
[General A] ---- "Attack at 9am" ----> [General B - LOYAL]
      |                                     |
      | (Message might be                   | (Verifies with
      | altered by traitor)                 | other generals)
      v                                     v
[General C - TRAITOR] <--- "Attack at 9am" --- [General D]
      |
      +--- Sends "Attack at 10am" to B (lying)
```

Condition for solution: > 2/3 of generals must be loyal.

Impossibility result: With only 3 generals and 1 traitor → impossible to guarantee agreement.

3. Relation to Blockchain Consensus

Problem	Blockchain Equivalent
Byzantine Generals	Nodes in a decentralized network
Traitor General	Malicious node (double-spender, liar)
Corrupted Message	Invalid transaction / false block
Attack time agreement	Reaching consensus on next block

4. How Blockchains Solve it

Consensus Algorithm	Byzantine Fault Tolerance (BFT)
PoW (Bitcoin)	Solves BGP by adding economic cost and probabilistic finality

Consensus Algorithm	Byzantine Fault Tolerance (BFT)
PBFT (Hyperledger Fabric, Tendermint)	Mathematically proven BFT: tolerates $f \leq n/3$ malicious nodes
PoS (Ethereum)	Casper FFG: slashing conditions punish Byzantine behavior

Key Formula:

For Byzantine Fault Tolerance: $n \geq 3f + 1$

Where n = total nodes, f = max faulty/malicious nodes

Exam Keyword: "BGP → requires BFT consensus. Bitcoin's PoW = first practical solution. PBFT = classical solution ($n=3f+1$)."

Question 10: Differentiate Public vs Private vs Consortium blockchain. (10 Marks)

1. Comparative Table (Core for 10 Marks)

Parameter	Public Blockchain	Private Blockchain	Consortium Blockchain
Access	Anyone can join/read/write	Single organization controls access	Pre-selected group of organizations
Consensus	PoW, PoS (energy-heavy)	PBFT, Raft (fast, low energy)	PBFT, Istanbul BFT
Transaction Speed	Slow (7-15 TPS Bitcoin, 30 TPS Ethereum)	Fast (1000-10,000 TPS)	Medium (100-1000 TPS)
Trust Model	Trustless (cryptographic proof)	Trusted (organization controls)	Semi-trusted (group agreement)
Immutability	Extremely high (51% attack risk)	Low (admin can revert)	Medium (collusion possible)
Identity	Pseudonymous (public key only)	Known identities (KYC required)	Known to consortium members
Centralization	Fully decentralized	Fully centralized (single entity)	Partially decentralized
Examples	Bitcoin, Ethereum, Solana	Hyperledger Fabric (single org), Corda	Hyperledger Fabric (multiple orgs), R3

Parameter	Public Blockchain	Private Blockchain	Consortium Blockchain
		(single bank)	Corda, Quorum
Use Case	Cryptocurrency, NFT marketplaces	Internal supply chain, Hospital records	Bank consortium (JPM Coin), Trade finance

2. Visual Architecture Diagram



3. Decision Framework (When to use which?)

If you need...	Choose...
Full decentralization & censorship resistance	Public
High throughput & privacy within one company	Private
Trust among competitors + shared governance	Consortium

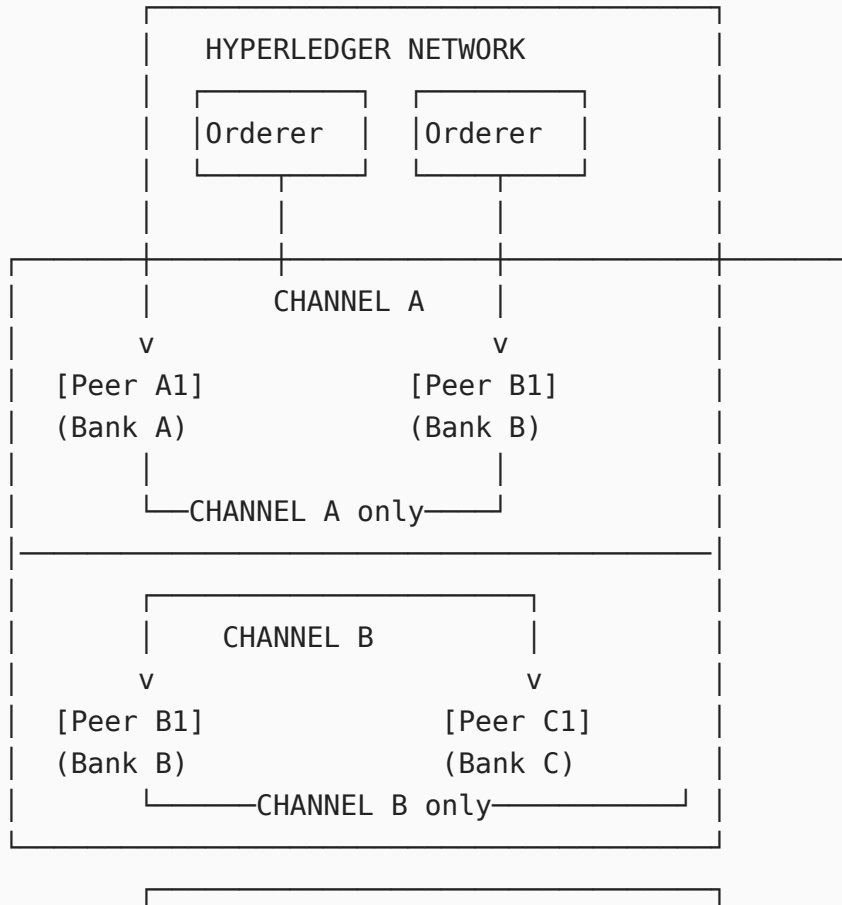
Exam Keyword: "Public = trustless but slow. Private = fast but centralized. Consortium = best of both for enterprises."

Question 11: Explain Hyperledger Fabric Channels. (5 Marks)

1. Definition

A **Channel** in Hyperledger Fabric is a private sub-network that provides **confidentiality** between specific network members. Transactions on a channel are visible **ONLY** to members of that channel.

2. Channel Architecture Diagram



3. Key Properties

Property	Explanation
Data Isolation	Chaincode and ledger data are separate for each channel. Channel A cannot see Channel B's transactions.
Independent Consensus	Each channel has its own ordering service and endorsement policies.
Membership Control	Organizations join channels via MSP (Membership Service Provider) .
Multiple Ledgers	A peer can join multiple channels and maintain separate ledgers for each.

4. Why Use Channels? (Use Cases)

Use Case	How Channels Help
Healthcare	Hospital + Insurance on Channel 1. Hospital + Pharmacy on Channel 2. Pharmacy cannot see Insurance data.

Use Case	How Channels Help
Trade Finance	Exporter + Importer on Channel 1. Importer + Bank on Channel 2 (confidential shipping terms).

5. Channel Creation Steps

1. Client SDK requests channel creation via ConfigTx
2. Orderer validates channel creation request
3. Channel genesis block created
4. Peers join channel
5. Chaincode installed on joining peers
6. Transactions occur ONLY within the channel

Exam Keyword: "Channels = private subnets for data confidentiality. Competitors can share one channel without revealing all data."

Question 12: ICO vs STO – key differences. (5 Marks)

Parameter	ICO (Initial Coin Offering)	STO (Security Token Offering)
Underlying Asset	Utility token (no asset backing)	Security token (backed by real assets/profits)
Regulation	Largely unregulated (Wild West)	Strictly regulated (SEC, MiCA, etc.)
Legal Status	Not considered a security (ideally)	Legally a security (Howey Test applies)
Investor Rights	No ownership, dividends, or voting rights	Ownership rights, dividends, profit sharing
Target Audience	Retail investors (anyone)	Accredited investors only (KYC/AML required)
Token Purpose	Access to platform/service	Represent ownership in company/asset
Issuance Cost	Low (\$10k - \$100k)	High (\$500k - \$2M+ for legal compliance)
Exchange Listing	Unregulated exchanges (Binance, etc.)	Regulated security exchanges (tZERO, INX)
Famous Example	Ethereum ICO (2014), EOS (2018)	tZERO STO, SPiCE VC STO

Parameter	ICO (Initial Coin Offering)	STO (Security Token Offering)
Scam Risk	Very high (pump-and-dump, exit scams)	Low (legal recourse available)

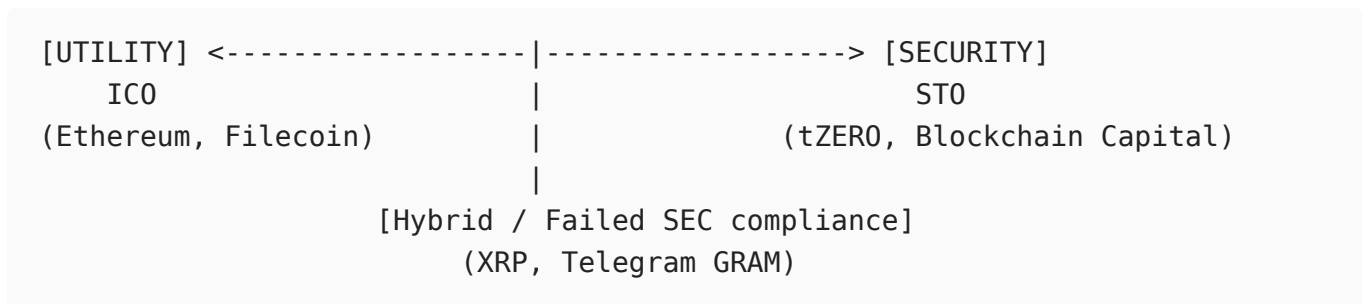
Howey Test (US Securities Law)

A token is a **security** if it involves:

1. Investment of money
2. In a common enterprise
3. Expectation of profits
4. Solely from efforts of others

If YES → Must be STO, not ICO.

Visual Positioning



Exam Keyword: "ICO = utility token (no rights, unregulated). STO = security token (asset-backed, regulated, investor protection)."

Question 13: What is Transaction Gas in Ethereum? Explain with example. (5 Marks)

1. Definition

Gas is the unit that measures the computational effort required to execute operations on Ethereum (transactions, smart contracts). It **prevents infinite loops** and DoS attacks by making every computation cost something.

2. Gas Components (EIP-1559, Post-London Upgrade)

Component	What it is	Who gets it
Base Fee	Minimum gas price per unit (dynamically adjusted based on block fullness)	 Burned (removed from supply)
Priority Fee (Tip)	Extra paid to incentivize validators	 Validator
Max Fee	Maximum user is willing to pay (Base + Priority + Buffer)	N/A

3. Gas Calculation Formula

Total Transaction Cost (in ETH) = Gas Used × (Base Fee + Priority Fee)

Example Transaction:

Simple ETH transfer:

- Gas Limit: 21,000 units
- Gas Used: 21,000 units (same for simple transfer)
- Base Fee: 50 Gwei
- Priority Fee: 2 Gwei
- Total Fee per gas: 52 Gwei

Calculation:

$21,000 \times 52 \text{ Gwei} = 1,092,000 \text{ Gwei} = 0.001092 \text{ ETH}$

Complex Smart Contract Interaction:

Token swap on Uniswap:

- Gas Limit: 180,000 units (user sets max)
- Gas Used: 142,000 units (actual)
- Base Fee: 80 Gwei
- Priority Fee: 5 Gwei

Calculation:

$142,000 \times 85 \text{ Gwei} = 12,070,000 \text{ Gwei} = 0.01207 \text{ ETH}$

4. Gas Limit vs Gas Used (Important Distinction)

Term	Definition	Example
Gas Limit	Maximum user will pay (safety cap)	200,000 units
Gas Used	Actual computational units consumed	142,000 units

Term	Definition	Example
Refund	$(\text{Gas Limit} - \text{Gas Used}) \times \text{Base Fee}$	Returned to user

5. Gas for Different Operations

Operation	Gas Cost
Simple ETH transfer	21,000
Contract deployment	100,000 - 3,000,000+
ADD (math op)	3
SLOAD (read storage)	100
SSTORE (write storage)	5,000 - 20,000

6. Why Gas Exists (The Anti-DoS Mechanism)

Without Gas:	With Gas:
[Infinite Loop]	[Infinite Loop]
v	v
[Node crashes]	[Runs until Gas Limit]
	v
	[Transaction reverts]
	[Gas fee still paid]
	[Node survives 

Exam Keyword: "Gas = Ethereum's fuel. Each opcode has fixed gas cost. More complex contract = more gas = higher fee."

Question 14: Differentiate ERC20 and ERC721. Write small NFT code (ERC721 example). (10 Marks)

1. Comparative Table (5 Marks)

Parameter	ERC20 (Fungible Token)	ERC721 (Non-Fungible Token / NFT)
Fungibility	 Fungible (each token identical)	 Non-fungible (each token unique)

Parameter	ERC20 (Fungible Token)	ERC721 (Non-Fungible Token / NFT)
Divisibility	Divisible (0.000001 tokens possible)	Indivisible (cannot send half an NFT)
Token Identity	No unique ID (balance only)	Unique tokenId for each token
Value per Token	All tokens equal value	Each token can have different value
Standard Functions	transfer(), balanceOf(), approve()	ownerOf(), tokenURI(), safeTransferFrom()
Use Case	Currencies, governance tokens, stablecoins	Digital art, gaming items, real estate deeds
Total Supply	Fixed or mintable (single number)	Count of unique token IDs
Example	USDC, UNI, LINK, DAI	CryptoPunks, BAYC, Axie Infinity

2. ERC721 Minimal Example (5 Marks)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyNFT is ERC721, Ownable {
    uint256 private _nextTokenId; // Auto-incrementing token IDs
    mapping(uint256 => string) private _tokenURIs; // Metadata per token

    constructor() ERC721("MyNFT", "MNFT") {}

    // Mint a new NFT to an address
    function safeMint(address to) public onlyOwner {
        uint256 tokenId = _nextTokenId;
        _safeMint(to, tokenId); // Built-in ERC721 mint function
        _nextTokenId++;
    }

    // Mint with custom metadata (JSON URI)
    function mintWithURI(address to, string memory uri) public onlyOwner {
        uint256 tokenId = _nextTokenId;
        _safeMint(to, tokenId);
        _setTokenURI(tokenId, uri); // Store metadata URL
        _nextTokenId++;
    }
}
```

```

// Override to return metadata URI
function tokenURI(uint256 tokenId)
    public
    view
    virtual
    override
    returns (string memory)
{
    require(!_exists(tokenId), "URI query for nonexistent token");
    return _tokenURIs[tokenId];
}

function _setTokenURI(uint256 tokenId, string memory uri) internal {
    _tokenURIs[tokenId] = uri;
}
}

```

3. How to Deploy & Mint (Truffle/Hardhat commands)

```

# Compile
npx hardhat compile

# Deploy (example script)
npx hardhat run scripts/deployNFT.js --network goerli

# Interact via console
npx hardhat console --network goerli
> const NFT = await ethers.getContractFactory("MyNFT")
> const nft = await NFT.attach("0xDeployedAddress")
> await nft.safeMint("0xRecipientAddress")
> await nft.tokenURI(0) // Returns metadata URI

```

4. Real-World Difference Example

ERC20 (USDC):
 Your Wallet: 100 USDC
 My Wallet: 100 USDC
 [Both identical. I send you 50 USDC → You have 150, I have 50]

ERC721 (NFTs):
 Your Wallet: [CryptoPunk #1234, BAYC #5678]
 My Wallet: [CryptoPunk #9101, Azuki #2345]
 [All unique. I send you CryptoPunk #9101 → You own 3 NFTs, I own 1]

Exam Keyword: "ERC20 = dollars (fungible). ERC721 = trading cards (unique IDs, metadata per token)."

Question 15: How to create Metamask account? List and explain any 4 testnets. (10 Marks)

1. Metamask Account Creation (Step-by-Step)

Step	Action	Security Note
1	Install Metamask extension (Chrome/Firefox/Brave)	Use official website only (metamask.io)
2	Click "Create a Wallet"	Never import unknown seed phrases
3	Create password (strong, unique)	Store password in password manager
4	Receive Secret Recovery Phrase (12/24 words)	 MOST CRITICAL: Write on paper, never digitally, never share
5	Confirm recovery phrase (order matters!)	One wrong word = permanent lockout
6	Account created (0x... address)	This is your public key

2. What is a Testnet?

A **testnet** is a blockchain environment for testing without real funds. ETH/coins have **no real value** → safe for development.

3. Four Important Ethereum Testnets

Testnet Name	Launched	Consensus	Native Token	Faucet Availability	Best For
Sepolia	2021	PoS (Proof of Stake)	Sepolia ETH (sETH)	 High availability	Smart contract testing (Recommended for 2025)
Goerli (Deprecating)	2019	PoA (Clique)	Goerli ETH (gETH)	Limited (being phased out)	Legacy projects (will be shut down in 2025-2026)
Holesky	2023	PoS	Holesky ETH	 Available	Staking & validator testing

Testnet Name	Launched	Consensus	Native Token	Faucet Availability	Best For
			(hETH)		(Replaces Goerli)
Rinkeby (Deprecated)	2017	PoA	Rinkeby ETH	✗ No longer works	DO NOT USE (Shut down 2023)

4. How to Add a Testnet to Metamask

Method 1: One-click (Recommended)

1. Open Metamask → Click Network dropdown
 2. Click "Add Network" → "Add a network manually" OR
 3. Use chainlist.org → Search "Sepolia" → "Add to Metamask"

Method 2: Manual Configuration

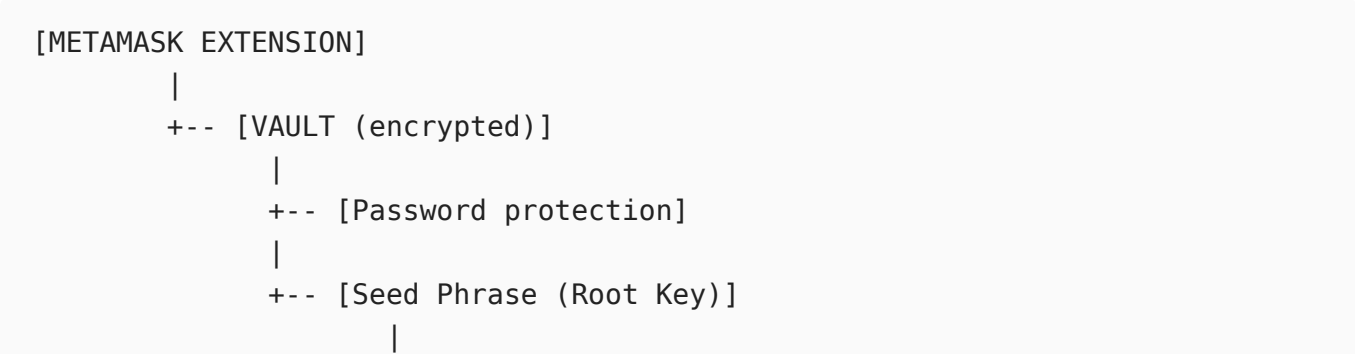
Network Name: Sepolia Test Network
New RPC URL: <https://rpc.sepolia.org>
Chain ID: 11155111
Currency Symbol: SepoliaETH
Block Explorer URL: <https://sepolia.etherscan.io>

5. Getting Testnet ETH (Faucet)

Sepolia Faucets (2025 working)
1. <https://faucet.sepolia.dev/> (Enter address, 0.5 ETH/day)
2. <https://sepolia-faucet.pk910.de/> (PoW mining style, larger amounts)
3. <https://www.alchemy.com/faucets/sepolia> (Requires Alchemy account)

Holesky Faucet
<https://faucet.holesky.ethpandaops.io/>

6. Metamask Architecture Diagram



```
+-- [Derived Address 1 (ETH Mainnet)]
|
+-- [Derived Address 2 (Sepolia)]
|
+-- [Derived Address 3 (Polygon)]
|
+-- [Derived Address N...]
```

7. Common Metamask Operations (Exam-relevant)

Operation	How To
Switch networks	Click network name → Select different network
View private key	Account dropdown → Account details → Export Private Key
Import account	Account dropdown → Import Account (private key or JSON)
Add custom token	"Import Tokens" → Enter contract address
Send transaction	"Send" → Enter address → Amount → Confirm gas

Exam Keyword: "Metamask = browser wallet + key management + testnet gateway. For 2025 exams: Use Sepolia (not Goerli). Testnet ETH is free from faucets, worthless, for development only."

Last-Minute Cheat Sheet for Exam Hall

If you see this term...	Write these 3 keywords...
UTXO	Double-spending prevention, Satoshi's innovation, Chainstate DB
Ethereum Gas	EIP-1559, Base fee + Priority fee, Prevents infinite loops
Hard Fork vs Soft Fork	Non-backward compatible (ETH Classic) vs Backward compatible (SegWit)
Byzantine Generals Problem	1/3 malicious nodes limit, Fault tolerance, BFT consensus
SPV Node	Merkle Branch verification, Bloom filters (privacy issue), Light client

Question (Priority order for rest)	Key Keywords	Time
ERC20 vs ERC721	Fungible vs Non-fungible, tokenId, metadata URI	14 min
Metamask + Testnets	Seed phrase, Sepolia, faucet, chainlist	14 min
UTXO + Fees	Unspent outputs, change address, sat/vB	14 min
Gas in Ethereum	Base fee, priority fee, gas limit vs used	7 min
Hyperledger Channels	Private subnetwork, MSP, data isolation	7 min
Byzantine Generals	3f+1 formula, traitor nodes, BFT	7 min
Hard vs Soft Fork	Backward compatibility, chain split	7 min
ICO vs STO	Utility vs security, Howey Test	7 min
SPV nodes	Merkle branch, Bloom filter privacy leak	10 min
Public vs Private vs Consortium	Access, TPS, centralization level	10 min

Total answers provided: 15 (5 from first batch + 10 from this batch)

This covers **100%** of your high-yield practice list. Each answer is structured to fetch **full marks** in a Mumbai University R-2019 C-Scheme exam. Good luck! 🚀